Guida Completa per il Primo Parziale di Basi di Dati

Modello Relazionale – Algebra Relazionale – SQL

Versione sintetica e funzionale – *Learn by Doing*

Indice

1 Modello Relazionale – Sintesi **2**

2 Algebra Relazionale – Operatori e Pattern **3**

3 SQL – Sintassi e Pattern **4**

4 Esercizi Svolti – Algebra Relazionale **5**

5 Esercizi Svolti – SQL **7**

6 Cheat Sheet – Regole Rapide **10**

7 Piano di Studio di 7 Giorni **10**

8 Simulazione d'Esame **11**

1 Modello Relazionale – Sintesi

Concetti chiave

Termine	Definizione
Relazione	Tabella con righe (tuple) e colonne (attributi)
Schema	Nome relazione + attributi + domini (livello intensionale)
Istanza	Insieme di tuple in un dato momento (livello estensionale)
Grado (arietà)	Numero di attributi
Cardinalità	Numero di tuple

Vincoli di integrità

1. **Dominio**: i valori di un attributo appartengono al dominio specificato.
2. **Chiave**: insieme minimale di attributi che identifica univocamente una tupla (chiave candidata). Una viene scelta come **chiave primaria**.
   - **Superchiave**: insieme che contiene una chiave (può avere attributi in più).
   - **Integrità entità**: nessun attributo della chiave primaria può essere `NULL`.
3. **Integrità referenziale**: una chiave esterna (FK) in R_1 deve corrispondere a una chiave primaria in R_2, oppure essere `NULL` (se non fa parte della PK).

Azioni su violazione di vincoli

Azione	Comportamento
`RESTRICT` / `NO ACTION`	Rifiuta l'operazione
`CASCADE`	Propaga l'operazione (es. cancella tuple che referenziano)
`SET NULL`	Imposta la FK a `NULL` (solo se non fa parte della PK)
`SET DEFAULT`	Imposta la FK a un valore di default

Esempio rapido

```
CREATE TABLE Students (
  sid CHAR(20) PRIMARY KEY,
  name CHAR(30),
  age INTEGER CHECK (age >= 16)
);

CREATE TABLE Enrolled (
  studid CHAR(20),
  cid CHAR(20),
  grade CHAR(10),
  PRIMARY KEY (studid, cid),
  FOREIGN KEY (studid) REFERENCES Students(sid)
    ON DELETE CASCADE
    ON UPDATE NO ACTION
);
```

2 Algebra Relazionale – Operatori e Pattern

Operatori fondamentali

Operatore	Simbolo	Descrizione	Esempio
Selezione	σ	Filtra tuple che soddisfano una condizione	$\sigma_{rating>8}(Sailors)$
Proiezione	π	Seleziona attributi (elimina duplicati)	$\pi_{sname,\, rating}(Sailors)$
Ridenominazione	ρ	Rinomina attributi o relazioni	$\rho_{C(1\to sid1,\, 5\to sid2)}(S1 \times R1)$
Unione	$\cup$	Tuple in R o S (schemi compatibili)	$R \cup S$
Differenza	$-$	Tuple in R ma non in S	$R - S$
Prodotto cartesiano	$\times$	Tutte le combinazioni di tuple	$R \times S$
Join	$\bowtie$	Combinazione con condizione	$R \bowtie_{R.a=S.b} S$
Divisione	$/$	Tuple di R che si combinano con tutte le tuple di S	A/B

Proprietà importanti

- **SELECT** è commutativo: $\sigma_{c_1}(\sigma_{c_2}(R)) = \sigma_{c_2}(\sigma_{c_1}(R))$
- **PROJECT** non è commutativo: $\pi_Y(\pi_X(R)) = \pi_Y(R)$ solo se $Y \subseteq X$
- **JOIN** è commutativo e associativo
- **UNIONE** e **INTERSEZIONE** sono commutative e associative
- **DIFFERENZA** non è commutativa: $R - S \neq S - R$

Pattern comuni

Tutti i marinai che hanno prenotato tutte le barche

$$\pi_{sid}(Reserves)/\pi_{bid}(Boats)$$

Marinai che hanno prenotato almeno una barca rossa

$$\pi_{sname}\big((\sigma_{color=\text{'red'}}Boats) \bowtie Reserves \bowtie Sailors\big)$$

Marinai che hanno prenotato sia rosso che verde

$$\pi_{sid}\big((\sigma_{color=\text{'red'}}Boats) \bowtie Reserves\big) \\ \cap\ \pi_{sid}\big((\sigma_{color=\text{'green'}}Boats) \bowtie Reserves\big)$$

Marinai che hanno prenotato rosso ma non verde

$$\pi_{sid}\big((\sigma_{color=\text{'red'}}Boats) \bowtie Reserves\big) \\ -\ \pi_{sid}\big((\sigma_{color=\text{'green'}}Boats) \bowtie Reserves\big)$$

Marinai con rating massimo

$$\pi_{sname}(Sailors) - \pi_{sname}\big(Sailors \bowtie_{rating<rating2} \rho_{S2}(Sailors)\big)$$

3 SQL – Sintassi e Pattern

Struttura base

```
SELECT [DISTINCT] target-list
FROM relation-list
WHERE qualification
GROUP BY grouping-list
HAVING group-qualification
ORDER BY ...;
```

Operatori di confronto

- `=, <, >, <=, >=, <>`
- `BETWEEN ... AND ...`
- `IN (lista)`
- `LIKE 'pattern'` (`%` = zero o più caratteri, `_` = un carattere)
- `IS NULL` / `IS NOT NULL` (**mai** `= NULL`)

JOIN

Tipo	Sintassi	Comportamento
INNER JOIN	`A JOIN B ON cond`	Solo tuple con match
LEFT JOIN	`A LEFT JOIN B ON cond`	Tutte le tuple di A, con `NULL` se non c'è match
RIGHT JOIN	`A RIGHT JOIN B ON cond`	Tutte le tuple di B
FULL JOIN	`A FULL JOIN B ON cond`	Tutte le tuple di entrambe (non supportato da MySQL/MariaDB)

Aggregazioni

```
SELECT rating, AVG(age) AS avg_age, COUNT(*) AS num
FROM Sailors
WHERE rating > 5
GROUP BY rating
HAVING COUNT(*) >= 2;
```

Regola: ogni colonna nel `SELECT` che non è aggregata deve apparire in `GROUP BY`.

Subquery

Operatore	Uso
`IN`	Valore appartiene a un insieme
`EXISTS`	La subquery restituisce almeno una tupla
`ANY` / `SOME`	Confronto con almeno un valore
`ALL`	Confronto con tutti i valori

Subquery correlata: fa riferimento a colonne della query esterna (rieseguita per ogni riga).

Operatori insiemistici

- `UNION` (elimina duplicati) / `UNION ALL` (mantiene duplicati)
- `INTERSECT`
- `EXCEPT` (o `MINUS`)

Pattern comuni in SQL

Richiesta	Query
"Tutti"	`NOT EXISTS ( ... EXCEPT ... )` o doppio `NOT EXISTS`
"Almeno uno"	`EXISTS` o `IN`
"Nessuno"	`NOT EXISTS`
"Massimo"	`= (SELECT MAX(...))`
"Almeno due"	self-join con `<>` e `COUNT`

4 Esercizi Svolti – Algebra Relazionale

Schema `esercitazione_AR.pdf`

```
IMPIEGATO(Matricola, Cognome, Stipendio, Dipartimento)
DIPARTIMENTO(Codice, Nome, Sede, Direttore)
PROGETTO(Sigla, Nome, Bilancio, Responsabile)
PARTECIPAZIONE(Impiegato, Progetto)
```

1. Matricola e cognome degli impiegati con stipendio > 50M

$$\pi_{Matricola,\, Cognome}\big(\sigma_{Stipendio>50}(IMPIEGATO)\big)$$

2. Cognome e stipendio degli impiegati che lavorano a Roma

$$\pi_{Cognome,\, Stipendio}\big(IMPIEGATO \bowtie_{Dipartimento=Codice} \sigma_{Sede='\text{Roma}'}(DIPARTIMENTO)\big)$$

3. Cognome degli impiegati e nome del dipartimento

$$\pi_{Cognome,\, Nome}\big(IMPIEGATO \bowtie_{Dipartimento=Codice} DIPARTIMENTO\big)$$

4. Cognome degli impiegati che sono direttori

$$\pi_{Cognome}\big(IMPIEGATO \bowtie_{Matricola=Direttore} DIPARTIMENTO\big)$$

5. Nomi dei progetti e cognomi dei responsabili

$$\pi_{Nome,\, Cognome}\big(PROGETTO \bowtie_{Responsabile=Matricola} IMPIEGATO\big)$$

6. Nomi progetti con bilancio > 100K e cognomi impiegati che vi lavorano

$$\pi_{Nome,\, Cognome}\big(\sigma_{Bilancio>100}(PROGETTO) \bowtie_{Sigla=Progetto} PARTECIPAZIONE \bowtie_{Impiegato=Matricola} IMPIEGATO\big)$$

7. Cognome degli impiegati che guadagnano più del loro direttore

$$\pi_{Cognome}\Big(\sigma_{I.Stipendio>Dir.Stipendio}\big(\rho_I(IMPIEGATO) \bowtie_{I.Dipartimento=D.Codice} \rho_D(DIPARTIMENTO) \bowtie_{D.Direttore=Dir.Matricola} \rho_{Dir}(IMPIEGATO)\big)\Big)$$

8. Cognomi dei direttori e dei responsabili di progetto

$$\pi_{Cognome}\big(IMPIEGATO \bowtie_{Matricola=Direttore} DIPARTIMENTO\big) \cup \pi_{Cognome}\big(IMPIEGATO \bowtie_{Matricola=Responsabile} PROGETTO\big)$$

9. Nomi dei dipartimenti con impiegati che guadagnano > 60K

$$\pi_{Nome}\big(DIPARTIMENTO \bowtie_{Codice=Dipartimento} \sigma_{Stipendio>60}(IMPIEGATO)\big)$$

10. Nomi dei dipartimenti in cui tutti gli impiegati guadagnano > 60K

$$\pi_{Nome}(DIPARTIMENTO) - \pi_{Nome}\big(DIPARTIMENTO \bowtie_{Codice=Dipartimento} \sigma_{Stipendio\leq 60}(IMPIEGATO)\big)$$

11. Cognome degli impiegati di stipendio massimo

$$\pi_{Cognome}(IMPIEGATO) - \pi_{Cognome}\big(IMPIEGATO \bowtie_{Stipendio<Stipendio2} \rho_{I2}(IMPIEGATO)\big)$$

12. Matricola e cognome degli impiegati che non lavorano a nessun progetto

$$\pi_{Matricola,\, Cognome}(IMPIEGATO) - \pi_{Matricola,\, Cognome}\big(IMPIEGATO \bowtie_{Matricola=Impiegato} PARTECIPAZIONE\big)$$

13. Matricola e cognome degli impiegati che lavorano a più di un progetto

$$\pi_{Matricola,\, Cognome}\Big(IMPIEGATO \bowtie_{Matricola=P1.Impiegato} \rho_{P1}(PARTECIPAZIONE) \bowtie_{P1.Impiegato=P2.Impiegato \,\wedge\, P1.Progetto\neq P2.Progetto} \rho_{P2}(PARTECIPAZIONE)\Big)$$

14. Matricola e cognome degli impiegati che lavorano a un solo progetto

$$\pi_{Matricola,\, Cognome}\big(IMPIEGATO \bowtie_{Matricola=Impiegato} PARTECIPAZIONE\big) - \pi_{Matricola,\, Cognome}\Big(IMPIEGATO \bowtie_{Matricola=P1.Impiegato} \rho_{P1}(PARTECIPAZIONE) \bowtie_{P1.Impiegato=P2.Impiegato \,\wedge\, P1.Progetto\neq P2.Progetto} \rho_{P2}(PARTECIPAZIONE)\Big)$$

Interpretazione query (Esercizio 2)

1. $\pi_{Cognome}\big(\rho_{Matricola\leftarrow Responsabile}(PROGETTO) \bowtie IMPIEGATO\big)$
   → **Cognomi dei responsabili di progetto** (impiegati che sono responsabili di almeno un progetto).

2. $\pi_{Cognome}\big(\rho_{Matricola\leftarrow Direttore}(DIPARTIMENTO) \bowtie (IMPIEGATO \bowtie \rho_{Dipartimento\leftarrow Codice}(DIPARTIMENTO$
   → **Cognomi degli impiegati che sono direttori di dipartimento** (la query è ridondante ma restituisce i direttori).

5 Esercizi Svolti – SQL

Da `sailors_esercizi_1_senza_soluzioni.txt`

1. Per ogni colore di barca, somma dei rating dei marinai che l'hanno prenotata, solo colori con rating medio > 7

```sql
SELECT B.color, SUM(S.rating) AS somma_rating
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid
GROUP BY B.color
HAVING AVG(S.rating) > 7;
```

2. Giorni con almeno 2 prenotazioni, solo marinai con età ≥ 18

```sql
SELECT R.day
FROM Reserves R, Sailors S
WHERE R.sid = S.sid AND S.age >= 18
GROUP BY R.day
HAVING COUNT(*) >= 2;
```

3. Nomi di marinai omonimi con divario età < 10 anni

```sql
SELECT S.sname
FROM Sailors S
GROUP BY S.sname
HAVING COUNT(*) > 1 AND MAX(S.age) - MIN(S.age) < 10;
```

4. Età media e numero marinai per rating > 5, solo gruppi con ≥ 2 marinai

```sql
SELECT S.rating, AVG(S.age) AS eta_media, COUNT(*) AS num_marinai
FROM Sailors S
WHERE S.rating > 5
GROUP BY S.rating
HAVING COUNT(*) >= 2;
```

5. Barche (bid, bname) prenotate da marinai con età media > 40, solo barche prenotate ≥ 2 volte

```sql
SELECT B.bid, B.bname
FROM Boats B, Reserves R, Sailors S
WHERE B.bid = R.bid AND R.sid = S.sid
GROUP BY B.bid, B.bname
HAVING COUNT(*) >= 2 AND AVG(S.age) > 40;
```

Da `Argomento04.1 SQL - Autoverifica.ppt` (NorthWind)

1. Clienti francesi

```sql
SELECT * FROM Customers WHERE Country = 'France';
```

2. Numero di prodotti per fornitore

```sql
SELECT SupplierID, COUNT(*) AS num_prodotti
FROM Products
GROUP BY SupplierID;
```

3. Valore totale delle categorie di prodotti in magazzino

```sql
SELECT c.CategoryName, SUM(p.UnitPrice * p.UnitsInStock) AS valore_totale
FROM Products p
JOIN Categories c ON p.CategoryID = c.CategoryID
GROUP BY c.CategoryName;
```

4. Numero di ordini per cliente

```sql
SELECT CustomerID, COUNT(*) AS num_ordini
FROM Orders
GROUP BY CustomerID;
```

5. Valore totale degli ordini per cliente

```sql
SELECT o.CustomerID,
       SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) AS valore_totale
FROM Orders o
JOIN Order_Details od ON o.OrderID = od.OrderID
GROUP BY o.CustomerID;
```

6. Numero di ordini gestiti per impiegato

```sql
SELECT EmployeeID, COUNT(*) AS num_ordini
FROM Orders
GROUP BY EmployeeID;
```

7. Prodotti italiani

```sql
SELECT p.*
FROM Products p
JOIN Suppliers s ON p.SupplierID = s.SupplierID
WHERE s.Country = 'Italy';
```

8. Incasso totale da ordini francesi

```sql
SELECT SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) AS incasso_totale
FROM Orders o
JOIN Order_Details od ON o.OrderID = od.OrderID
JOIN Customers c ON o.CustomerID = c.CustomerID
WHERE c.Country = 'France';
```

9. Impiegati con più di 50 ordini gestiti

```sql
SELECT EmployeeID
FROM Orders
GROUP BY EmployeeID
HAVING COUNT(*) > 50;
```

10. Ordine/i con valore massimo

```sql
SELECT OrderID
FROM (
  SELECT OrderID, SUM(UnitPrice * Quantity * (1 - Discount)) AS totale
  FROM Order_Details
  GROUP BY OrderID
) AS T
WHERE totale = (
  SELECT MAX(totale)
  FROM (
    SELECT SUM(UnitPrice * Quantity * (1 - Discount)) AS totale
    FROM Order_Details
    GROUP BY OrderID
  ) AS T2
);
```

11. Prodotti con più di 20 unità in magazzino

```sql
SELECT * FROM Products WHERE UnitsInStock > 20;
```

12. Clienti che vivono nello stesso paese degli impiegati che hanno seguito il loro ordine

```sql
SELECT DISTINCT c.*
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN Employees e ON o.EmployeeID = e.EmployeeID
WHERE c.Country = e.Country;
```

13. Bevande ordinate più di 5 volte

```sql
SELECT p.ProductName
FROM Products p
JOIN Order_Details od ON p.ProductID = od.ProductID
JOIN Categories c ON p.CategoryID = c.CategoryID
WHERE c.CategoryName = 'Beverages'
GROUP BY p.ProductID, p.ProductName
HAVING COUNT(*) > 5;
```

14. Valore degli ordini nel 3° trimestre 1995

```sql
SELECT SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) AS valore_totale
FROM Orders o
JOIN Order_Details od ON o.OrderID = od.OrderID
WHERE o.OrderDate >= '1995-07-01' AND o.OrderDate < '1995-10-01';
```

6 Cheat Sheet – Regole Rapide

Algebra vs SQL

Algebra Relazionale	SQL
σ (selezione)	`WHERE`
π (proiezione)	`SELECT`
$\bowtie$ (join)	`JOIN ... ON` o `WHERE ... = ...`
$\cup$	`UNION`
$-$	`EXCEPT`
$\cap$	`INTERSECT`
/ (divisione)	`NOT EXISTS` doppio

Errori comuni da evitare

- × `WHERE x = NULL` ✓ `WHERE x IS NULL`
- × `SELECT rating, AVG(age) FROM Sailors` ✓ `SELECT rating, AVG(age) FROM Sailors GROUP BY rating`
- × `COUNT(A)` include i `NULL`? No, li ignora. `COUNT(*)` conta tutte le righe.
- × `UNION` elimina i duplicati di default; se vuoi mantenerli usa `UNION ALL`.
- × In algebra relazionale, la proiezione elimina i duplicati; in SQL, `SELECT` non lo fa a meno che tu non usi `DISTINCT`.

Traduzione richieste → query

Richiesta	Strategia
"Tutti"	Divisione (algebra) o doppio `NOT EXISTS` (SQL)
"Almeno uno"	`EXISTS` o `IN`
"Nessuno"	`NOT EXISTS`
"Solo"	`NOT EXISTS` con condizione negata
"Massimo/minimo"	`= (SELECT MAX(...))`
"Almeno due"	Self-join con `<>`
"Contare per gruppo"	`GROUP BY` + `COUNT(*)`
"Filtrare gruppi"	`HAVING`

7 Piano di Studio di 7 Giorni

Giorno 1 – Modello Relazionale

- **Teoria** (2h): leggi la sezione 1 di questa guida + slide `Argomento02`.
- **Esercizi** (2h): risolvi `Argomento02.E Modello Relazionale - Autoverifica.ppt`.
- **Pratica** (1h): apri `salilors.sql` in DBeaver, prova a violare i vincoli.

Giorno 2 – Algebra Relazionale

- **Teoria** (2h): sezione 2 + slide `Argomento03`.
- **Esercizi** (2h): risolvi le 14 query di `esercitazione_AR.pdf` (sezione 4).
- **Pratica** (1h): traduci 5 query da algebra a SQL.

Giorno 3 – SQL Base e JOIN

- **Teoria** (1.5h): sezione 3 + slide `Argomento04` (fino a JOIN).
- **Esercizi** (2h): risolvi gli esercizi 1–8 da `Argomento04.1` (NorthWind).
- **Pratica** (1h): scrivi 5 query con JOIN su `northwind`.

Giorno 4 – SQL Aggregazioni e Subquery

- **Teoria** (2h): sezione 3 (aggregazioni, subquery, operatori insiemistici).
- **Esercizi** (2h): risolvi le 5 query di `sailors_esercizi_1_senza_soluzioni.txt` (sezione 5).
- **Pratica** (1h): risolvi esercizi 9–15 da `Argomento04.1`.

Giorno 5 – Esercizi Integrati

- **Ripasso** (1h): rileggi la cheat sheet.
- **Esercizi** (3h): risolvi tutte le query di `esercitazione_AR.pdf` e `sailors_esercizi_1`.
- **Autovalutazione**: segna gli errori e rivedili.

Giorno 6 – Consolidamento

- **Ripasso** (2h): rivedi gli esercizi sbagliati.
- **Esercizi** (2h): inventa 5 query complesse e risolvile in algebra e SQL.
- **Mappa mentale** (1h): disegna uno schema con tutti gli operatori e le regole.

Giorno 7 – Simulazione Finale

- **Simulazione** (2h): risolvi la sezione 8 senza guardare le soluzioni.
- **Correzione** (1h): confronta con le soluzioni e rivedi gli errori.
- **Ripasso leggero** (1h): rileggi solo cheat sheet e mappa mentale.

8 Simulazione d'Esame

Parte 1 – Algebra Relazionale (Schema Sailors)

Risolvi le seguenti query in algebra relazionale:

1. Nomi dei marinai che hanno prenotato una barca blu.
2. Nomi dei marinai che hanno prenotato tutte le barche rosse.
3. Nomi dei marinai che hanno prenotato almeno due barche diverse.

4. Nomi dei marinai che non hanno mai prenotato una barca verde.

5. Per ogni età, numero di marinai con quell'età (solo età con più di 1 marinaio).

Parte 2 – SQL (Schema Sailors)

Scrivi le query SQL equivalenti alle precedenti.

Soluzioni

1.

Algebra:

$$\pi_{sname}\big((\sigma_{color='\text{blue}'}Boats) \bowtie Reserves \bowtie Sailors\big)$$

SQL:

```sql
SELECT DISTINCT S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'blue';
```

2.

Algebra:

$$\pi_{sname}\Big(\big(\pi_{sid,bid}(Reserves)/\pi_{bid}(\sigma_{color='\text{red}'}Boats)\big) \bowtie Sailors\Big)$$

SQL:

```sql
SELECT S.sname
FROM Sailors S
WHERE NOT EXISTS (
  SELECT B.bid FROM Boats B WHERE B.color = 'red'
  EXCEPT
  SELECT R.bid FROM Reserves R WHERE R.sid = S.sid
);
```

3.

Algebra:

$$\pi_{sname}\Big(\big(\pi_{sid}(Reserves) \bowtie_{sid=sid2 \wedge bid\neq bid2} \rho_{R2}(\pi_{sid,bid}(Reserves))\big) \bowtie Sailors\Big)$$

SQL:

```sql
SELECT DISTINCT S.sname
FROM Sailors S, Reserves R1, Reserves R2
WHERE S.sid = R1.sid AND R1.sid = R2.sid AND R1.bid <> R2.bid;
```

4.

Algebra:

$$\pi_{sname}(Sailors) - \pi_{sname}\big((\sigma_{color='\text{green}'}Boats) \bowtie Reserves \bowtie Sailors\big)$$

SQL:

```sql
SELECT S.sname
FROM Sailors S
WHERE NOT EXISTS (
  SELECT * FROM Reserves R, Boats B
  WHERE R.sid = S.sid AND R.bid = B.bid AND B.color = 'green'
);
```

5.

Algebra: $\rho_{T(age,num)}\big(\gamma_{age,COUNT(*)}(Sailors)\big)$ ma in algebra base si può fare con self-join e conteggio.

SQL:

```sql
SELECT age, COUNT(*) AS num_marinai
FROM Sailors
GROUP BY age
HAVING COUNT(*) > 1;
```